

Graph Attention Networks Final Report

Introduction

The paper we reimplemented is Graph Attention Networks (GAT) by Petar Veličković et al. It's a novel neural network architecture that operates on graph-structured data. It uses masked self-attention layers to improve prior methods such as graph convolution neural networks. The GAT model allows the nodes to attend to the neighboring nodes by different weights without requiring any costly matrix operations like inversion or prior knowledge of the graph structure. The datasets we used are Cora, Citeseer, Pubmed, and PPI.

Chosen Results

The specific results we chose to reproduce from the paper were all of the GAT results for both the transductive and inductive learning tasks. They represent the results of the main focus of the paper. For clarification purposes, we reproduced GAT results where the neighbors of each node had their attention scores calculated and, thus, we did not reproduce the results of Const-GAT which does not calculate different attention scores for different neighboring nodes. For baselines, we only included MLP and GCN.

Methodology

For our reimplementation, we created a Jupyter Notebook to run our code in Google Colab. We created the architecture of the neural network according to the paper's instructions. There are many hidden settings the paper used but did not mention explicitly, so we did a lot of trials and errors in the process. It is impossible to include all of them, so we summarized the trials and errors that are fairly significant into three experiments.

In experiment 1, we made three modifications to the original code specified by the paper. The first is a different patience update rule. The original paper's rule is 100 consecutive epochs with neither validation loss nor validation accuracy improvement. Ours were 100 consecutive epochs with no validation accuracy improvement, so it's a stricter stopping rule. The second and third are fewer maximum training epochs and manual garbage collection. These three modifications were to prevent out-of-memory errors. For the manual garbage collection, we combined PyTorch's dedicated method for clearing the cache of NVIDIA GPUs with Python's garbage collection module called gc. Experiment 2 is an exploration beyond the original paper. It is about exploring the effect of stacking more GAT layers together. In experiment 2, we used the exact settings as experiment 1 except that we stacked 4 GAT layers for all datasets, whereas the original paper used 2 layers for the Cora, Citeseer, and Pubmed datasets and 3 layers for the PPI dataset. Experiment 3 addresses several implementation discrepancies identified in experiment 1. We divide this experiment into two variants, 3A and 3B, based on early stopping and checkpoint selection strategy. In experiment 3A (original paper style early Stopping), we restored the original early stopping logic, where both validation accuracy and validation loss are tracked, and patience is reset when either one improves. In experiment 3B (loss-based early stopping), we modified the checkpoint policy to save the model whenever validation loss decreases, even if validation accuracy does not improve simultaneously. This prevents potentially optimal model states from being discarded.

Across experiment 3, we implemented the following corrections: correcting LeakyReLU slope from PyTorch default (0.01) to 0.2, as specified in the original paper. For weight initialization, we replaced the Kaiming uniform with a Xavier uniform for all learnable weight matrices (including $W_{(a_1, a_2)}$, and residual projections) and initialized all

biases to zero. For residual connections, we ensured projection layers include trainable bias terms. Finally, we added missing self-loops before adjacency construction, aligning with the treatment used in transductive datasets. One thing to note is that we tested multiple random seeds in experiments 2 and 3, unlike experiment 1.

Results & Analysis

In experiment 1, our results were slightly lower than the original paper except for our Citeseer dataset results. Our hypothesis for this was that this was because we only tested one predefined seed. Thus, we tested for multiple seeds in experiments 2 and 3. In experiment 2, we tried stacking more GAT layers, but the results got even worse (see Table 1 below). For experiment 3A, the score is higher than experiment 1 for the Cora and Citeseer datasets and lower in the Pubmed and PPI datasets. For experiment 3B, we use a stopping rule only based on validation loss, but surprisingly, the scores improved upon all tasks (see Table 1 below).

Reflections

The takeaway across experiment 1, 3A, and 3B is that validation loss is a better checkpoint-selection signal than accuracy or micro-F1 scores. Experiment 3A only saves when accuracy and loss both improve, which filters lucky snapshots on the Cora and Citeseer datasets (about a point gain). However, on the PPI dataset with wide, unregularized, intermittent loss spikes, it rejects saves at those spikes and rolls back to an undertrained checkpoint (0.970 to 0.966). Experiment 3B selects on loss alone and wins on all four tasks because loss is continuous and aligned with the training objective. Experiment 3A underperforms on the PPI dataset not because adding loss is wrong, but because it drags the better signal down by coupling it with noisier accuracy. From experiment 2, we know that stacking more GAT layers lowers model performance, by inducing overfitting.

In conclusion, this project demonstrates that the performance of Graph Attention Networks depends not only on the expressive power of attention itself, but also on subtle training dynamics such as checkpoint selection, regularization, and architectural depth. Our results highlight an important lesson in deep learning: stronger models are not achieved simply by stacking more layers, but by balancing representation power with stable optimization and generalization.

References

Veličković, P., Cucurull, G., Casanova, A., Romero, A., Lio, P., & Bengio, Y. (2017). Graph attention networks. *arXiv preprint arXiv:1710.10903*.
Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI magazine*, 29(3):93, 2008
Used ChatGPT and Claude code to explain the code and paper and helped with the coding.

Appendices

Method	Transductive (Test Accuracy)			Inductive
	Cora	Citeseer	Pubmed	PPI (Micro F1)
MLP	55.10%	46.50%	71.40%	0.422
GCN-64	$81.4 \pm 0.5\%$	$70.9 \pm 0.5\%$	$79.0 \pm 0.3\%$	0.934 ± 0.006
GAT (paper)	$83.0 \pm 0.7\%$	$72.5 \pm 0.7\%$	$79.0 \pm 0.3\%$	0.973 ± 0.002
GAT (Experiment 1)	82.00%	71.80%	77.70%	0.97
GAT (Experiment 2)	$80.7 \pm 0.36\%$	$69.1 \pm 0.85\%$	$77.03 \pm 0.32\%$	0.9345 ± 0.0090
GAT (Experiment 3A)	$83.06 \pm 0.65\%$	$72.18 \pm 0.95\%$	$77.68 \pm 0.49\%$	0.9657 ± 0.0046
GAT (Experiment 3B)	$83.46 \pm 0.96\%$	$72.46 \pm 0.5\%$	$78.3 \pm 0.29\%$	0.9704 ± 0.0034

Table 1: Reimplementation results of Graph Attention Networks (GAT) on transductive (Cora, Citeseer, Pubmed) and inductive (PPI) benchmarks.